

# AI architecture design

AI is a technology that machines use to imitate intelligent human behavior. Machines can use AI to do the following tasks:

- Analyze data to create images and videos.
- Analyze and synthesize speech.
- Verbally interact in natural ways.
- Make predictions and generate new data.

You can incorporate AI into applications to do functions or make decisions that traditional logic or processing can't handle effectively. As an architect who designs solutions, you need to learn about the AI and machine learning landscape and how you can integrate Azure solutions into your workload design.

## Get started

Azure Architecture Center provides example architectures, architecture guides, architectural baselines, and ideas that you can apply to your scenario. Workloads that use AI and machine learning components should follow the Azure Well-Architected Framework [AI workloads](#) guidance. This guidance includes principles and design guides that influence AI and machine learning workloads across the five architecture pillars. Implement those recommendations in the scenarios and content in the Azure Architecture Center.

## AI concepts

AI concepts encompass a wide range of technologies and methodologies that machines use to do tasks that typically require human intelligence. The following sections provide an overview of key AI concepts.

## Algorithms

*Algorithms or machine learning algorithms* are pieces of code that help people explore, analyze, and find meaning in complex datasets. Each algorithm is a finite set of unambiguous step-by-step instructions that a machine can follow to achieve a specific goal. The goal of a machine

learning model is to establish or discover patterns that humans can use to make predictions or categorize information. An algorithm might describe how to check whether a pet is a cat, dog, fish, bird, or lizard. A more complicated algorithm might describe how to identify a written or spoken language, analyze its words, translate them into a different language, and then check the translation for accuracy.

Choose an algorithm family that best suits your task. Evaluate the different algorithms within the family to find the appropriate fit for your workload. For more information, see [Machine learning algorithms](#) .

## Machine learning

*Machine learning* is an AI technique that uses algorithms to create predictive models. These algorithms parse data fields and learn from the patterns within data to generate models. The models can then make informed predictions or decisions based on new data.

The predictive models are validated against known data, measured by performance metrics for specific business scenarios, and then adjusted as needed. This process of learning and validation is called *training*. Through periodic retraining, machine learning models improve over time.

In your workload design, you might use machine learning if your scenario includes past observations that you can reliably use to predict future situations. These observations can be universal truths, like computer vision that detects one form of animal from another. Or these observations can be specific to your situation, like computer vision that detects a potential assembly mistake on your assembly lines based on past warranty claim data.

For more information, see [Machine learning overview](#) .

## Deep learning

*Deep learning* is a type of machine learning that can learn through its own data processing. Like machine learning, it also uses algorithms to analyze data. But it analyzes data by using artificial neural networks that have many inputs, outputs, and layers of processing. Each layer can process the data in a different way. The output of one layer becomes the input for the next. Deep learning uses this process to create more complex models than traditional machine learning can create.

Deep learning requires a large investment to generate highly customized or exploratory models. You might consider other solutions in this article before you add deep learning to your workload.

For more information, see [Deep learning overview](#) .

## Generative AI

*Generative AI* trains models to generate original content based on many forms of content, including natural language, computer vision, audio, or image input. By using generative AI, you can describe a desired output in everyday language, and the model can respond by creating appropriate text, image, and code. Examples of generative AI applications include Microsoft 365 Copilot and Microsoft Foundry.

- [Copilot](#) is primarily a user interface (UI) that helps you write code, documents, and other text-based content. It's based on popular models from OpenAI and Anthropic and is integrated into a wide range of Microsoft applications and user experiences.
- [Foundry](#) is a development platform as a service (PaaS) that provides access to agent hosting and a catalog of language models, including the following options:
  - GPT-5.2 (OpenAI)
  - Claude (Anthropic)
  - Phi (Microsoft)
  - Grok (xAI)

You can adapt these models to the following specific tasks:

- Content generation
- Content summarization
- Image understanding
- Semantic search
- Natural language to code translation
- Video generation
- Speech to speech

## Language models

*Language models* are a subset of generative AI that focus on natural language processing tasks, like text generation and sentiment analysis. These models represent natural language based on the probability of words or sequences of words that occur in a given context.

Conventional language models are used in supervised settings for research purposes. These models are trained on well-labeled text datasets for specific tasks. Pretrained language models provide an easy way to start using AI. They're more widely used in recent years. These models are trained on large-scale text collections from the internet via deep learning neural networks. You can fine-tune them on smaller datasets for specific tasks.

The number of parameters, or *weights*, determine the size of a language model. Parameters influence how the model processes input data and generates an output. During training, the model adjusts the weights to minimize the difference between its predictions and the actual data. This process is how the model learns parameters. The more parameters a model has, the more complex and expressive it is. But it's also more computationally expensive to train and use.

Small language models usually have fewer than 10 billion parameters, and large language models have more than 10 billion parameters. For example, the Microsoft Phi-4 model family includes the following versions:

- Phi-4-Mini, which has 3.8 billion parameters
- Phi-4-Multimodal-instruct, which has 5.6 billion parameters
- Phi-4 (the base model), which has 14 billion parameters

For more information, see the [language model catalog](#) .

## Copilots

The availability of language models led to new ways to interact with applications and systems by using digital copilots and connected, domain-specific agents. *Copilots* are generative AI assistants that integrate into applications, often as chat interfaces. They provide contextualized support for common tasks in those applications.

[Microsoft 365 Copilot](#) integrates with a wide range of Microsoft applications and user experiences. It's based on an open architecture where non-Microsoft developers can create their own plug-ins to extend or customize the user experience by using Copilot. Partner developers can also create their own copilots by using the same open architecture.

For more information, see the following resources:

- [Adopt, extend, and build Copilot experiences across the Microsoft Cloud](#)
- [Microsoft Copilot Studio overview](#)

- [Foundry overview](#)

## Retrieval-augmented generation

*Retrieval-augmented generation (RAG)* is an architecture pattern that augments the capabilities of a language model, like ChatGPT, that's trained only on public data. You can use this pattern to add a retrieval system that provides relevant grounding data in the context with the user request. An information retrieval system provides control over grounding data that a language model uses when it formulates a response. RAG architecture helps you scope generative AI to content that's sourced from vectorized documents, images, and other data formats. RAG isn't limited to vector search storage. You can use any data store technology.

For more information, see [Design and develop a RAG solution](#) and [Choose an Azure service for vector search](#). Use [Foundry IQ knowledge bases](#) for grounding data that Foundry agents need as a turnkey approach to RAG.

## Agent-based architecture

Agents are more than just code that calls language models to respond to user prompts. They can autonomously do tasks, make decisions, and interact with other systems. You can design agents to handle specific tasks or operate in complex environments, which makes them suitable for many applications. Multi-agent architecture lets you break complex problems into specialized agents that coordinate to produce a solution.

Tools like [Microsoft Agent Framework](#) and [Foundry workflows](#) can help you build agent-based architectures.

For information about how to coordinate multiple agents in complex AI scenarios, see [AI agent orchestration patterns](#).

## Foundry Tools

By using [Foundry Tools](#) , developers and organizations can use ready-made, prebuilt, and customizable APIs and models to create intelligent, market-ready, and responsible applications. Use cases include natural language processing for conversations, search, monitoring, translation, speech, vision, and decision-making.

For more information, see the following resources:

- [Choose a Foundry Tools technology](#)
- [Foundry Tools overview](#)
- [Choose a natural language processing technology in Azure](#)

## AI language models

*Language models*, like the OpenAI GPT models, are powerful tools that can generate natural language across different domains and tasks. To choose a model, consider factors like data privacy, ethical use, accuracy, and bias.

[Phi open models](#) are small, less compute-intensive models for generative AI solutions. A small language model might be more efficient, interpretable, and explainable than a large language model.

When you design a workload, you can use language models as a hosted solution behind a metered API. For many small language models, you can host language models in-process or at least on the same compute as the consumer. When you use language models in your solution, consider your choice of language model and its available hosting options to help ensure an optimized solution for your use case.

## AI development platforms and tools

The following AI development platforms and tools can help you build, deploy, and manage machine learning and AI models.

### Azure Machine Learning

Azure Machine Learning is a machine learning service that you can use to build and deploy models. Machine Learning provides web interfaces and SDKs for you to train and deploy your machine learning models and pipelines at scale. Use these capabilities with open-source Python frameworks like PyTorch, TensorFlow, and scikit-learn.

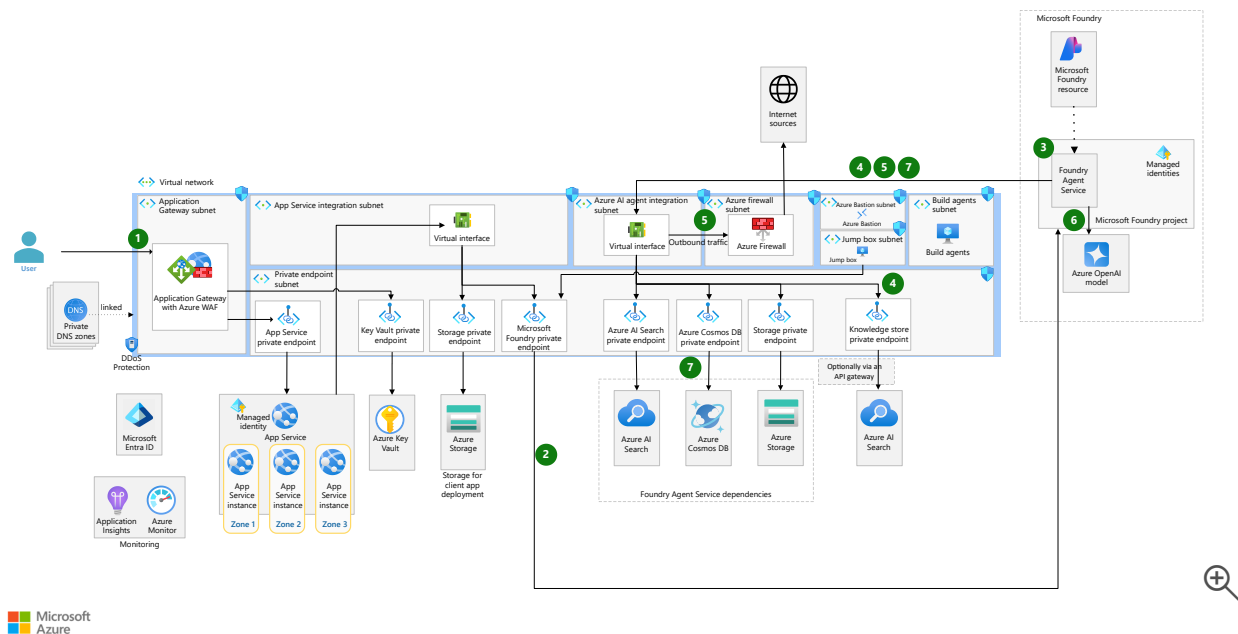
For more information, see the following resources:

- [Compare Microsoft machine learning products and technologies](#)
- [Machine Learning documentation](#)

- [What is Machine Learning?](#)

## AI and Machine learning reference architectures for Azure

- [Baseline Foundry chat reference architecture in an Azure landing zone](#)
- [Baseline Foundry chat reference architecture](#) describes how to build an end-to-end chat architecture by using the OpenAI GPT models in Foundry. It incorporates grounding via enterprise data sources to enrich responses with contextual information.



## Automated machine learning

*Automated machine learning (AutoML)* is the process of automating the time-consuming, iterative tasks of machine learning model development. Data scientists, analysts, and developers can use AutoML to build machine learning models that have high scale, efficiency, and productivity while sustaining model quality.

For more information, see the following resources:

- [What is AutoML?](#)
- [Train a classification model by using AutoML in Machine Learning studio](#)
- [Set up AutoML experiments in Python](#)
- [Install and set up the CLI](#)

# MLflow

Machine Learning workspaces are MLflow-compatible, which means that you can use a Machine Learning workspace the same way that you use an MLflow server. This compatibility provides the following advantages:

- Machine Learning doesn't host MLflow server instances but can use the MLflow APIs directly.
- You can use a Machine Learning workspace as your tracking server for any MLflow code, whether or not it runs in Machine Learning. You need to set up MLflow to point to the workspace where the tracking should occur.
- You can run training routines that use MLflow in Machine Learning without making any changes.

For more information, see [MLflow and Machine Learning](#) and [MLflow](#) .

## Generative AI tools

- [Foundry](#) provides a platform to help you experiment, develop, and deploy generative AI apps and APIs responsibly. Use the [Foundry portal](#) to find Foundry Tools, foundation models, a playground, and resources to help you fine-tune, evaluate, and deploy AI models and AI agents.

[Foundry Agent Service](#) hosts agents that you define. These agents connect to a foundation model in the AI model catalog and optionally your own custom knowledge stores or APIs. You can define these agents declaratively or Foundry can containerize and host them.

- [Copilot Studio](#) extends Copilot in Microsoft 365. You can use Copilot Studio to build custom copilots for internal and external scenarios. Use an authoring canvas to design, test, and publish copilots. You can easily create generative AI-enabled conversations, provide greater control of responses for existing copilots, and accelerate productivity by using automated workflows.

## Data platforms for AI

The following platforms provide solutions for data movement, processing, ingestion, transformation, real-time analytics, and reporting.



# Microsoft Fabric

Microsoft Fabric is an end-to-end analytics and data platform for enterprises that require a unified solution. Workload teams can use data within Fabric. The platform covers data movement, processing, ingestion, transformation, real-time event routing, and report building. It provides a suite of services, including Fabric Data Engineer, Fabric Data Factory, Fabric Data Science, Fabric Real-Time Intelligence, Fabric Data Warehouse, and Fabric Databases.

Fabric integrates separate components into a cohesive stack. Instead of relying on different databases or data warehouses, you can centralize data storage by using OneLake. AI capabilities are embedded within Fabric, which eliminates the need for manual integration.

For more information, see the following resources:

- [What is Fabric?](#)
- [Learning path: Get started with Fabric](#)
- [Foundry Tools in Fabric](#)
- [Use Azure OpenAI in Fabric with REST API](#)
- [Use Fabric for generative AI: A guide to building and improving RAG systems](#)
- [Build custom AI applications with Fabric: Implement RAG for enhanced language models](#)

## Copilots in Fabric

You can use Copilot and other generative AI features to transform and analyze data, generate insights, and create visualizations and reports in Fabric and Power BI. You can build your own copilot or choose one of the following prebuilt copilots:

- [Copilot in Fabric](#)
- [Copilot for Data Science and Data Engineer](#)
- [Copilot for Data Factory](#)
- [Copilot for Data Warehouse](#)
- [Copilot for Power BI](#)
- [Copilot for Real-Time Intelligence](#)

## Data agent in Fabric

Data agent in Fabric is a feature that you can use to build your own conversational Q&A systems by using generative AI. A Fabric data agent makes data insights easier to use and more

actionable for everyone in your organization.

For more information, see the following resources:

- [Fabric data agent overview](#)
- [Create data agent](#)
- [Example of a data agent](#)
- [Difference between a Fabric data agent and a copilot](#)

## Apache Spark-based data platforms for AI

Apache Spark is a parallel processing framework that supports in-memory processing to boost the performance of big data analytic applications. Spark provides basic building blocks for in-memory cluster computing. A Spark job can load and cache data into memory and query it repeatedly, which is faster than disk-based applications, like Hadoop.

### Spark in Fabric

Fabric Runtime is an Azure-integrated platform based on Spark that you can use to implement and manage data engineering and data science experiences. Fabric Runtime combines key components from internal and open-source sources, which provides a comprehensive solution.

Fabric Runtime has the following key components:

- **Spark** is an open-source distributed computing library that you can use for large-scale data processing and analytics tasks. Spark provides a versatile platform for data engineering and data science experiences.
- **Delta Lake** is an open-source storage layer that integrates atomicity, consistency, isolation, and durability (ACID) transactions and other data reliability features with Spark. Integrated within Fabric Runtime, Delta Lake enhances data processing capabilities and helps ensure data consistency across multiple concurrent tasks.
- **Default-level packages for Java, Scala, Python, and R** are packages that support diverse programming languages and environments. These packages are automatically installed and configured, so developers can apply their preferred programming languages for data processing tasks.

Fabric Runtime is built on an open-source operating system that provides compatibility with different hardware configurations and system requirements.

For more information, see [Spark runtimes in Fabric](#).

## Azure Databricks Runtime for Machine Learning

[Azure Databricks](#) is a Spark-based analytics platform that includes workflows and an interactive workspace for collaboration between data scientists, engineers, and business analysts.

You can use [Databricks Runtime for Machine Learning](#) to start a Databricks cluster that has all the libraries required for distributed training. This feature provides an environment for machine learning and data science. It has multiple popular libraries, including TensorFlow, PyTorch, Keras, and XGBoost. It also supports distributed training via Horovod.

For more information, see the following resources:

- [Azure Databricks documentation](#)
- [Machine learning capabilities in Azure Databricks](#)
- [Deep learning overview for Azure Databricks](#)

## Spark in Azure HDInsight

[Spark in Azure HDInsight](#) is the Microsoft implementation of Spark in the cloud. Spark clusters in HDInsight are compatible with Azure Storage and Azure Data Lake Storage, so you can use HDInsight Spark clusters to process data that you store in Azure.

[SynapseML](#) is the Microsoft machine learning library for Spark. This open-source library adds many deep learning and data science tools, networking capabilities, and production-grade performance to the Spark ecosystem.

For more information, see the following resources:

- [SynapseML features and capabilities](#)
- [HDInsight overview](#)
- [Tutorial: Build an Spark machine learning application in HDInsight](#)
- [Spark best practices on HDInsight](#)
- [Set up HDInsight Spark cluster settings](#)
- [Create an Spark machine learning pipeline on HDInsight](#)

# Data storage for AI

You can use the following platforms to efficiently store, use, and analyze large volumes of data.

## Fabric OneLake

OneLake in Fabric is a unified and logical data lake that you can tailor to your entire organization. It's the central hub for all analytics data and is included with every Fabric tenant. OneLake in Fabric is built on the foundation of Data Lake Storage.

OneLake in Fabric provides the following benefits:

- Supports structured and unstructured file types
- Stores all tabular data in Delta-Parquet format
- Provides a single data lake within tenant boundaries that's governed by default
- Supports the creation of workspaces within a tenant so that your organization can distribute ownership and access policies
- Supports the creation of different data items, like lakehouses and warehouses, where you can use data

For more information, see [OneLake, the OneDrive for data](#).

## Data Lake Storage

Data Lake Storage is a single, centralized repository where you can store your structured and unstructured data. Use a data lake to quickly and easily store, use, and analyze a wide variety of data in a single location. You don't need to conform your data to fit an existing structure. Instead, you can store your data in its raw or native format, usually as files or as binary large objects, or blobs.

Data Lake Storage provides file system semantics, file-level security, and scale. Because these capabilities are built on Azure Blob Storage, you also get low-cost, tiered storage that has high availability and disaster recovery capabilities.

Data Lake Storage uses the infrastructure of Storage to create a foundation to build enterprise data lakes on Azure. Data Lake Storage can service multiple petabytes of information while sustaining hundreds of gigabits of throughput so that you can manage massive amounts of data.

For more information, see the following resources:

- [Introduction to Data Lake Storage](#)
- [Tutorial: Data Lake Storage, Azure Databricks, and Spark](#)

## Data processing for AI

You can use the following tools to prepare data for machine learning and AI applications. Ensure that your data is clean and structured so that you can use it for advanced analytics.

### Fabric Data Factory

You can use Fabric Data Factory to ingest, prepare, and transform data from multiple data sources, like databases, data warehouses, lakehouses, and real-time data streams. This feature can help you meet your data operations requirements when you design workloads.

Data Factory supports code solutions and no-code or low-code solutions:

- Use [data pipelines](#) to create workflow capabilities at cloud scale. Use the select-and-move interface to build workflows that can refresh your data flow, move petabyte-size data, and define control-flow pipelines.
- Use [dataflows](#) as a low-code interface to ingest data from hundreds of data sources and transform it by using over 300 data transformations.

For more information, see [Data Factory end-to-end scenario: Introduction and architecture](#).

### Azure Databricks

You can use the Databricks Data Intelligence Platform to write code to create a machine learning workflow by using feature engineering. *Feature engineering* is the process of transforming raw data into features that you can use to train machine learning models. The Databricks Data Intelligence Platform includes key features that support feature engineering:

- **Data pipelines** ingest raw data, create feature tables, train models, and do batch inference. When you use feature engineering in Unity Catalog to train and log a model, the model is packaged with feature metadata. When you use the model for batch scoring or online

inference, it automatically retrieves feature values. The caller doesn't need to know about the values or include logic to look up or join features to score new data.

- **Model and feature serving endpoints** are instantly available and provide milliseconds of latency.
- **Monitoring** helps ensure the performance and accuracy of data and models.

You can also use [Mosaic AI Vector Search](#) to store and retrieve embeddings. Embeddings are crucial for applications that require similarity searches, like RAG, recommendation systems, and image recognition.

For more information, see [Serve data for machine learning and AI](#).

## Data connectors for AI

Azure Data Factory and Azure Synapse Analytics pipelines support many data stores and formats via copy, data flow, look up, get metadata, and delete activities. To see the available data store connectors, supported capabilities and their corresponding configurations, and generic Open Database Connectivity options, see [Azure Data Factory and Azure Synapse Analytics connector overview](#).

## Custom AI

Custom AI solutions help you address specific business needs and challenges. The following sections provide an overview of different tools and services that you can use to build and manage custom AI models.

## Azure Machine Learning

Azure Machine Learning is a cloud service for accelerating and managing the machine learning project life cycle. Machine learning professionals, data scientists, and engineers can use this service in their workflows to train and deploy models and manage machine learning tasks.

Machine Learning provides the following capabilities:

- **Algorithm selection:** Some algorithms make specific assumptions about data structure or desired results. Choose an algorithm that fits your needs so that you can get more useful

results, more accurate predictions, and faster training times. For more information, see [How to choose algorithms for Machine Learning](#).

- **Hyperparameter tuning or optimization:** You can use this manual process to find hyperparameter configurations that result in the best performance. This optimization incurs significant computational costs. *Hyperparameters* are adjustable parameters that provide control in the model training process. For example, you can choose the number of hidden layers and the number of nodes in each layer of neural networks. Model performance depends heavily on hyperparameters.

You can use Machine Learning to automate hyperparameter tuning and run experiments in parallel to efficiently optimize hyperparameters.

For more information, see the following resources:

- [Hyperparameter tune a model](#)
  - [Upgrade hyperparameter tuning to SDK v2](#)
  - [Learning path: Tune hyperparameters by using Machine Learning](#)
- **Model training:** You can iteratively use an algorithm to create or teach models. After you train the models, you can use them to analyze data and make predictions.

The following steps happen during the training phase:

1. A quality set of known data is tagged so that individual fields are identifiable.
2. An algorithm that's configured to make a particular prediction receives the tagged data.
3. The algorithm outputs a model that captures the patterns that it identified in the data. The model uses a set of parameters to represent these patterns.

The following steps happen during validation:

1. Fresh data is tagged and used to test the model.
2. The algorithm is adjusted as needed and possibly does more training.
3. The testing phase uses real-world data without any tags or preselected targets. If the model's results are accurate, it's ready for use and can be deployed.

For more information, see the following resources:

- [Train models by using Machine Learning](#)
- [Tutorial: Train a model in Machine Learning](#)
- [Distributed training with Machine Learning](#)
- **AutoML:** This process automates the time-consuming, iterative tasks of machine learning model development. It can significantly reduce the time that it takes to produce production-ready machine learning models. AutoML can assist with model selection, hyperparameter tuning, model training, and other tasks, without requiring extensive programming or domain knowledge.

You can use AutoML when you want Machine Learning to use a specific target metric to train and tune a model. You don't need data science expertise to find problems in an end-to-end machine learning pipeline.

Machine learning professionals and developers across industries can use AutoML to do the following tasks:

- Implement machine learning solutions without extensive programming or machine learning knowledge.
- Save time and resources.
- Apply data science best practices.
- Provide agile problem-solving.

For more information, see [AutoML overview](#).

- **Scoring:** This process, also called *prediction*, uses a trained machine learning model to generate values based on new input data. The values, or scores, can represent predictions of future values, but they might also represent a likely category or outcome.

For more information, see the following resources:

- [Score model](#)
- [Deploy models for scoring in batch endpoints](#)
- **Feature engineering and featurization:** Training data consists of rows and columns. Each row is an observation or record, and the columns of each row are the features that describe each record. Typically, you choose the features that best characterize the patterns in the data to create predictive models.



Although you can use many of the raw data fields to train a model, you might need to create other engineered features that provide information to differentiate patterns in the data more easily. This process is called *feature engineering*, where you use domain knowledge of the data to create features that help machine learning algorithms learn better.

In Machine Learning, data-scaling and normalization techniques are applied to make feature engineering easier. Collectively, these techniques and feature engineering are called *featurization* in AutoML experiments. For more information, see [Data featurization in AutoML](#).

## Microsoft Foundry

In Foundry, you can use a process called *fine-tuning* to tailor models to your personal datasets. This customization step optimizes the service by providing the following benefits:

- Higher quality results compared to [prompt engineering](#) only
- The ability to train on more examples than a model's maximum request context limit typically permits
- Token savings because of shorter prompts
- Lower-latency requests, particularly when you use smaller models

For more information, see the following resources:

- [Customize a model by using fine-tuning](#)
- [Azure OpenAI GPT-4o-mini fine-tuning tutorial](#)
- [Baseline Foundry chat reference architecture](#)

## Foundry Tools for custom AI

[Foundry Tools](#) provides features to build custom AI models and applications. The following sections provide an overview of these key features.

### Custom speech

[Custom speech](#) is a feature of Azure Speech. You can use custom speech to evaluate and improve the accuracy of speech recognition for your applications and products. Use a custom speech model for real-time speech to text, speech translation, and batch transcription.

By default, speech recognition uses a universal language model as a base model. This model is trained with Microsoft-owned data and reflects commonly used spoken language. The base model is pretrained with dialects and phonetics that represent common domains. When you make a speech recognition request, the most recent base model for your supported language is used by default. The base model works well in most speech recognition scenarios.

You can use a custom model to augment the base model. For example, you can improve the recognition of domain-specific vocabulary that's specific to an application by providing text data to train the model. You can also improve recognition for specific audio conditions of an application by providing audio data, including reference transcriptions.

If the data follows a pattern, you can use structured text to train a model. You can specify custom pronunciations and customize display text formatting by using custom inverse text normalization, custom rewrite, and custom profanity filtering.

## Custom translator

[Custom translator](#) is a feature of [Azure Translator](#). Enterprises, app developers, and language service providers can use custom translator to build customized neural machine translation (NMT) systems. The customized translation systems integrate into existing applications, workflows, and websites.

You can use this feature to build and publish custom translation systems to and from English. Custom translator supports more than three dozen languages that map directly to the languages for NMT. For a complete list of languages, see [Translator language support](#).

Custom translator provides the following features.

 **Expand table**

| Feature                                                           | Description                                                                                                                 |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Apply NMT technology</a>                              | Apply NMT from the custom translator to improve your translation.                                                           |
| <a href="#">Build systems that know your business terminology</a> | Customize and build translation systems by using parallel documents that use the terminology in your business and industry. |
| <a href="#">Use a dictionary to build your models</a>             | Train a model by using only dictionary data if you don't have a training dataset.                                           |

| Feature                                           | Description                                                                                                                     |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">Collaborate with others</a>           | Collaborate with your team by sharing your work with different people.                                                          |
| <a href="#">Use your custom translation model</a> | Use your custom translation model anytime by using your existing applications or programs via Microsoft Translator Text API V3. |

## Custom models

[Azure Document Intelligence](#) uses advanced machine learning technology to identify documents, detect and extract information from forms and documents, and return the extracted data in a structured JSON output. Use Document Intelligence to take advantage of prebuilt or pretrained document analysis models or trained standalone custom models.

[Document Intelligence custom models](#) include custom classification models for scenarios where you need to identify the document type before you invoke the extraction model. You can pair a classification model with a custom extraction model to analyze and extract fields from forms and documents that are specific to your business. Combine standalone custom extraction models to create [composed models](#).

## Custom analyzer

[Azure Content Understanding](#) uses generative AI to process or ingest many types of content, including documents, images, videos, and audio, into a user-defined output format. Content Understanding comes with prebuilt analyzers for common content types and scenarios.

[Custom analyzer](#) is a feature of Content Understanding that creates custom analyzers tailored to your specific content processing needs. You can define custom extraction rules and entity recognition patterns to meet your business requirements.

## Custom AI tools

Prebuilt AI models are useful and increasingly flexible, but the best way to optimize AI is to tailor a model to your specific needs. Two primary tools to create custom AI models are generative AI and traditional machine learning.

## Azure Machine Learning studio

[Azure Machine Learning studio](#) is a cloud service for accelerating and managing the machine learning project life cycle. Machine learning professionals, data scientists, and engineers can use it in their workflows to train and deploy models and manage machine learning tasks.

- Build and train Machine Learning models by using any type of compute, including Spark and GPUs for cloud-scale, large AI workloads.
- Run AutoML and use the select-and-move UI for low-code Machine Learning.
- Implement end-to-end Machine Learning tasks and repeatable pipelines.
- Use the responsible AI dashboard for bias detection and error analysis.
- Orchestrate and manage prompt engineering and language model flows.
- Deploy models via REST API endpoints, real-time inference, and batch inference.
- Use hub workspaces to share compute, quota, security, and connectivity to enterprise resources, while centralizing governance for IT. Set up a hub once, and then create secure workspaces directly from the studio for each project. Use hubs to manage your team's work in the studio and the [Foundry portal](#) .

## Foundry

[Foundry](#) helps you efficiently build and deploy custom generative AI applications by using Azure AI capabilities.

- Build together as one team. Your Foundry account provides enterprise-grade security and a collaborative environment that includes shared resources and connections to pretrained models, data, and compute.
- Organize your work. Your Foundry project helps you save state so that you can iterate from the first idea to the first prototype and first production deployment. Easily invite others to collaborate with you.
- Use your preferred development platform and frameworks, including GitHub, Visual Studio Code, Agent Framework, Semantic Kernel, and AutoGen.
- Discover and benchmark models from a broad catalog.

- Set up models as a service (MaaS) through serverless APIs and hosted fine-tuning.
- Incorporate multiple models, data sources, and modalities.
- Build RAG by using your protected enterprise data, without the need for fine-tuning.
- Orchestrate and manage prompt engineering and large language model flows.
- Design and safeguard apps and APIs via configurable filters and controls.
- Evaluate model responses by using built-in and custom evaluation flows.
- Deploy AI innovations to the Azure-managed infrastructure to provide continuous monitoring and governance across environments.
- Continuously monitor deployed apps for safety, quality, and token consumption in production.

## Foundry Agent Service in the Foundry portal

Foundry Agent Service is a tool that you can use to create AI agents by using a no-code and nondeterministic approach. The agents are exposed as microservices on the Foundry account.

Each agent connects to a foundation model from the Foundry Models catalog. Agents can optionally connect to your custom private knowledge stores or public data. Agents can also invoke tools to call into custom code and do tasks.

## Custom AI code languages

The core concept of AI is the use of algorithms to analyze data and generate models to describe, or score, it in useful ways. Developers and data scientists, and sometimes other algorithms, use programming code to write algorithms. Two of the most popular programming languages for AI development are Python and R.

**Python** is a general-purpose, high-level programming language. It has a simple syntax that emphasizes readability. You don't need to run a compilation step. Python has a large standard library, and it supports the ability to add modules and packages. This feature encourages modularity and lets you expand capabilities when needed. A large and growing ecosystem of AI and machine learning libraries exist for Python, including many in Azure.

For more information, see the following resources:

- [Python on Azure](#)
- [Azure for Python developers](#)
- [Machine Learning SDK for Python](#)
- [Introduction to machine learning with Python and notebooks](#)
- [Scikit-learn open-source machine learning library for Python](#)
- [PyTorch open-source Python library](#)
- [TensorFlow open-source symbolic math library](#)
- [Tutorial: Apply machine learning models in Azure Functions with Python and TensorFlow](#)

**R** is a language and environment for statistical computing and graphics. You can use it for everything from mapping broad social and marketing trends online to developing financial and climate models.

Microsoft supports the R programming language and provides multiple options for R developers to run their code in Azure.

For more information, see [Use R interactively on Machine Learning](#).

For general information about custom AI on Azure, see the following resources:

- [Microsoft AI on GitHub: Samples, reference architectures, and best practices](#)
- [Machine Learning SDK for Python](#)
- [Machine Learning examples repository](#)
- [Train R models by using the Machine Learning CLI v2](#)

## Customer stories

Many industries apply AI in innovative and inspiring ways. Consider the following customer case studies and success stories:

- [NSF enables life-saving treatments to get to patients faster with Azure AI](#)
- [PIMCO boosts client service with an AI-powered search platform built on Azure AI](#)
- [Legrand and Azure OpenAI: Powering smarter solutions with AI-driven tools](#)
- [C.H. Robinson overcomes decades-old barriers to automate the logistics industry by using Azure AI](#)

[Browse more AI customer stories](#)

## General information about Microsoft AI

Learn more about Microsoft AI, and stay up to date with related news:

- [Microsoft AI](#)
- [AI learning hub](#)
- [Azure AI](#)
- [Microsoft AI news](#)
- [Microsoft AI on GitHub: Samples, reference architectures, and best practices](#)

## Next step

- [AI workloads on Azure](#)

## Related resource

- [Architecture diagrams and technology descriptions for AI solutions reference architectures](#)

---

Last updated on 01/13/2026